

Team 7 CV-12 Guide

Camera Image Rectification + Camera Input & Preprocessing Integration

Ubuntu WSL2 • ROS 2 Jazzy • Gazebo • image_proc • cv_bridge • YOLO11n-seg ONNX

Task mapping

The Planner card title “Set up camera image rectification” comes from the earlier rectification work item. In the current Sprint 2 task board, this work belongs under CV-12 — Camera Input & Preprocessing Integration. CV-12 covers the Gazebo overhead camera input, ROS Image→OpenCV conversion, colour handling, letterbox/input preparation, and mapping results back to source-image coordinates. This guide keeps rectification as the first concrete subtask.

This document is designed to be used directly by the team member completing the camera-rectification task. It starts from the supplied team7_project_bundle.zip, gives the exact terminal workflow, explains what success should look like, and includes failure/debug branches. It also identifies which Python files should be kept unchanged, which file can optionally be replaced, and one small validation node that can be added without interfering with CV-17 localisation work.

Source baseline used for this guide: Sprint 2 CV task board, the supplied Team 7 project bundle, the existing CV-14 WSL/interface guide, and the camera metadata already observed from the current Team 7 Gazebo overhead camera.

1. What CV-12 is actually about

CV-12 is a camera-input and preprocessing integration task. The goal is not to retrain the model and not to redesign localisation. The goal is to make the camera stream entering the detector technically correct, reproducible and ready for both Gazebo and a later physical camera.

1. Verify the overhead camera image and CameraInfo topics are alive and consistent.
2. Create a rectified image stream using the standard ROS 2 image_proc rectification node.
3. Confirm that cv_bridge converts the ROS image into the BGR format expected by the current Python pipeline.
4. Feed the rectified full-resolution image into the existing ONNX preprocessing pipeline.
5. Confirm the detector still letterboxes to 640×640 internally and maps masks/boxes back to the source image size.
6. Record evidence showing the raw topic, CameraInfo, rectified topic, detector input parameter and final detections.
7. Keep the implementation configurable so the same detector can later receive a physical-camera rectified topic without rewriting inference.py.

Important boundary with CV-17

Camera rectification and pixel-to-world localisation are different tasks. For CV-12, do not replace geometry.py and do not rework the TF/world projection. A rectified pixel should eventually be interpreted using the rectified camera model (projection matrix P when it differs from K), but the localisation implementation itself belongs to CV-17.

2. What is already in the supplied .zip

Component	Current bundle state	What CV-12 should do
Top camera image	/top_camera/image	Treat as the unrectified/raw input stream.
Top camera metadata	/top_camera/camera_info	Use as calibration input for image_proc.
Overhead resolution	1280 × 960	Keep rectified output at the same source resolution.

Component	Current bundle state	What CV-12 should do
Gazebo camera format	R8G8B8	cv_bridge may convert to BGR8 for OpenCV.
Model input	1×3×640×640 float32 RGB	Keep the current inference.py preprocessing.
Detector topic parameter	image_topic is already configurable	Point it to /top_camera/image_rect at startup.
Camera info parameter	camera_info_topic is already configurable	Keep /top_camera/camera_info unless the camera driver changes.
inference.py	Letterbox + RGB conversion + normalisation + unscale already implemented	Do not rewrite for this task.

2.1 Current simulation calibration already observed

The current Gazebo overhead camera has already produced the following calibration values during integration testing. The team member should still run the verification command locally rather than hard-code these values.

```
[text]
width: 1280
height: 960
distortion_model: plumb_bob
D: [0, 0, 0, 0, 0]
fx = 1108.7654113769531
fy = 1108.7654113769531
cx = 640.0
cy = 480.0
frame_id: overhead_camera/mount/overhead_camera
```

Therefore the current Gazebo camera is already an ideal pinhole camera with zero lens distortion. Running image_proc is still the correct architecture, but the rectified image is expected to look the same as the raw image. That is a success condition, not evidence that rectification failed.

2.2 Why rectification is still worth setting up

A physical camera normally has non-zero radial/tangential distortion. The ROS CameraInfo message carries the information needed to remove it:

- K — intrinsic matrix for the unrectified image.
- D — distortion coefficients.
- R — rectification rotation.
- P — projection matrix for the rectified image.

The standard image_proc::RectifyNode consumes the image and CameraInfo together and publishes image_rect. This makes the downstream detector independent of whether the camera is simulated or physical.

3. Recommended pipeline architecture

```
[text]
Gazebo overhead camera
|
+--> /top_camera/image -----+
|                               |
+--> /top_camera/camera_info -----+
|                               |
|                               v
|                               image_proc::RectifyNode
|                               |
|                               v
|                               /top_camera/image_rect
|                               |
|                               v
|                               cv_bridge -> BGR8
|                               |
|                               v
|                               inference.py preprocess
```

```

1280x960 -> 640x640 letterbox
|
v
YOLO11n-seg ONNX
|
v
masks/boxes restored to 1280x960
|
v
/phone_detections

```

The important design choice is that rectification happens before model preprocessing. Letterboxing should not be treated as a substitute for rectification: letterboxing changes image size/aspect placement; rectification corrects the camera projection model.

4. Ubuntu WSL2 setup

4.1 Windows prerequisites

Use Windows 11 with WSL2 and Ubuntu 24.04. ROS 2 Jazzy is the target distribution used by the supplied Team 7 bundle. If the Ubuntu WSL distribution has been moved from C: to D:, that is fine; inside Ubuntu, continue to work from the Linux filesystem under `~` rather than directly from `/mnt/c` or `/mnt/d` when running ROS/Gazebo.

4.2 Install / verify Ubuntu 24.04

```

[powershell]
wsl --install -d Ubuntu-24.04

```

If Ubuntu already exists, do not reinstall it. Open Ubuntu and verify the distribution with:

```

[bash]
lsb_release -a

```

4.3 Install / verify ROS 2 Jazzy

```

[bash]
source /opt/ros/jazzy/setup.bash
ros2 --help
printenv ROS_DISTRO

```

Expected ROS_DISTRO: jazzy. If ROS 2 is not installed, use the same Jazzy installation procedure as the CV-14 guide before continuing.

4.4 Copy the Team 7 bundle into the Linux filesystem

```

[bash]
cp -r "/mnt/d/OnTrack Tasks/Ubuntu, WSL, Gazebo/team7_project_bundle" ~/
cd ~/team7_project_bundle

```

Change the Windows path to the actual location of the extracted bundle. If the bundle is already at `~/team7_project_bundle`, skip this copy.

```

[bash]
chmod +x run_team7_sim.sh run_team7_perception.sh \
team7_sim_mark2_v1.0/scripts/*.sh \
team7_sim_mark2_v1.0/overlay/setup_overlay.sh

```

4.5 Install the rectification dependency

```

[bash]

```

```
sudo apt update
sudo apt install -y ros-jazzy-image-proc ros-jazzy-rqt-image-view
```

Verify that the standalone rectifier executable exists:

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 pkg executables image_proc | grep rect
```

A normal installation should show `rectify_node`. If nothing is returned, see Debug D3 later in this guide.

4.6 Build the perception workspace

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
colcon build --symlink-install
source install/setup.bash
```

Do not clean-build unnecessarily

A normal colcon build is enough before any source change. Use `rm -rf build install log` only when a stale generated interface or old Python installation is clearly causing a problem.

5. Baseline checks before starting rectification

Do not debug `image_proc` until the raw camera itself is proven healthy. Start the simulation first.

```
[bash]
bash ~/team7_project_bundle/run_team7_sim.sh
```

The supplied launcher should eventually report that the simulation is up and open the Team 7 camera viewer. In another terminal:

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep top_camera
ros2 topic info /top_camera/image -v
ros2 topic info /top_camera/camera_info -v
```

Then capture one message from each input:

```
[bash]
ros2 topic echo /top_camera/image --once --field width
ros2 topic echo /top_camera/image --once --field height
ros2 topic echo /top_camera/image --once --field encoding
ros2 topic echo /top_camera/image --once --field header
ros2 topic echo /top_camera/camera_info --once
```

Record at least width, height, encoding, `frame_id`, `distortion_model`, `D`, `K` and `P` from the output.

5.1 Minimum success gate before rectification

Check	Pass condition
<code>/top_camera/image</code> exists	At least one live publisher.
<code>/top_camera/camera_info</code> exists	At least one live publisher.
Image size	1280 × 960 in the current bundle.
Image is not black	Scene/table/phone area visible.

Check	Pass condition
CameraInfo calibration	K is non-zero; width/height match image.
Distortion model	Current Gazebo: plumb_bob, D all zero.

If the raw image is black or missing

Stop here. Rectification cannot repair a missing/black sensor feed. Fix the simulation/camera bridge first; the supplied launch scripts already contain the WSL/VirtualBox software-rendering workaround.

6. Start image_proc rectification

Leave the simulation running. Open a new Ubuntu terminal and run:

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 run image_proc rectify_node --ros-args \
  -r image:=/top_camera/image \
  -r camera_info:=/top_camera/camera_info \
  -r image_rect:=/top_camera/image_rect
```

Keep this terminal open. The rectifier must stay alive while the detector is consuming /top_camera/image_rect.

6.1 Verify the new topic

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 topic info /top_camera/image_rect -v
ros2 topic hz /top_camera/image_rect
ros2 topic echo /top_camera/image_rect --once --field header
```

The rectified stream should have approximately the same rate as the source camera. The Gazebo sensor is configured for 2 Hz, although software rendering can make the observed rate lower on some WSL/VM machines.

6.2 Visual verification

```
[bash]
ros2 run rqt_image_view rqt_image_view
```

In rqt_image_view, select /top_camera/image and then /top_camera/image_rect. In the current Gazebo setup the two should look essentially identical because D is zero.

7. Connect the rectified stream to the existing detector

The current onnx_phone_detector_node.py already declares image_topic and camera_info_topic as ROS parameters. Therefore the safest CV-12 integration does not require replacing the detector source file.

```
[bash]
source /opt/ros/jazzy/setup.bash
cd ~/team7_project_bundle/team7_perception_ws
source install/setup.bash

ros2 run team7_perception onnx_phone_detector --ros-args \
  -p image_topic:=/top_camera/image_rect \
  -p camera_info_topic:=/top_camera/camera_info
```

Use this direct ros2 run command instead of run_team7_perception.sh for the test, because the supplied shell script starts the node with its default parameters and does not currently forward extra ROS arguments.

7.1 Prove the detector is actually using the rectified topic

```
[bash]
ros2 param get /onnx_phone_detector image_topic
ros2 param get /onnx_phone_detector camera_info_topic
ros2 topic info /top_camera/image_rect -v
ros2 topic echo /phone_detections
```

Expected image_topic: /top_camera/image_rect. Keep a screenshot showing the parameter and a live /phone_detections message.

8. Verify the preprocessing chain

The current inference.py already performs the main preprocessing expected by the ONNX model. For CV-12, the job is to verify it rather than rewrite it.

Stage	Current code behaviour	CV-12 verification
ROS Image → OpenCV	CvBridge imgmsg_to_cv2(..., desired_encoding="bgr8")	No exception; output frame has shape 960×1280×3.
Letterbox	Aspect-preserving resize to 640×640 with grey padding	No stretching of source aspect ratio.
Colour	BGR → RGB before ONNX	Channels are correct for model input.
Normalisation	uint8 → float32 / 255.0	Input range is 0–1.
Tensor layout	HWC → CHW + batch dimension	Shape becomes 1×3×640×640.
Box recovery	unscale_box removes padding and divides by scale	Coordinates return to source image bounds.
Mask recovery	Mask is cropped out of letterbox then resized to original H×W	Mask aligns with 1280×960 source/rectified image.

8.1 Important coordinate convention after rectification

When the detector consumes /top_camera/image_rect, the “original image coordinates” produced by inference.py mean the original rectified source image — currently 1280 × 960 — not the 640 × 640 model tensor. In the current Gazebo camera, raw and rectified coordinates coincide because distortion is zero.

Future physical-camera note

With non-zero distortion, raw and rectified pixels are not the same geometric rays. If CV-17 consumes detections from a rectified image, it should interpret those pixels with the rectified projection matrix P (when P differs from K). Do not silently use raw K for rectified pixels on a real camera.

9. Python files: what should and should not be replaced

This section is intentionally conservative because CV-17 localisation work may be developed in parallel. The rectification task should avoid overwriting unrelated geometry/integration work.

File in original .zip	Action for CV-12	Reason
team7_perception/inference.py	KEEP	Already contains correct letterbox, BGR→RGB, normalisation, mask reconstruction and coordinate recovery.
team7_perception/onnx_phone_detector_node.py	KEEP for required task	image_topic and camera_info_topic are already configurable; start it with ROS parameter overrides.
team7_perception/geometry.py	DO NOT TOUCH	Owned by pixel-to-world/CV-17 work, not rectification.
team7_perception/setup.py	OPTIONAL REPLACE	Only needed if the optional validation helper below should be runnable with ros2 run.
team7_perception/camera_rectification_check.py	OPTIONAL ADD	Small evidence/debug helper that compares raw and rectified frames without modifying detector logic.

Merge safety

If a newer CV-17 branch has already changed `onnx_phone_detector_node.py` or `geometry.py`, do not replace those files with older code from this guide or from the original bundle. CV-12 can be completed entirely through `image_proc` + topic/parameter configuration.

9.1 Optional one-line detector default change

Only after the rectifier is always started with the perception stack, the detector default can be changed from raw to rectified input:

```
[python]
# original
self.declare_parameter("image_topic", "/top_camera/image")

# optional CV-12 default
self.declare_parameter("image_topic", "/top_camera/image_rect")
```

This is not required for task completion. Keeping the raw topic as the default and overriding the parameter at launch is safer until CV-19 packages the full launch/configuration.

10. Optional validation helper: `camera_rectification_check.py`

The following node is useful if the assignee wants stronger evidence than two rqt windows. It subscribes to raw and rectified images, matches them by timestamp, logs the CameraInfo calibration, calculates a pixel-difference statistic, and saves one raw/rectified/difference image set.

```
[python]
#!/usr/bin/env python3
"""Small CV-12 validation node for raw vs rectified camera images.

This node does not perform the rectification itself. ROS 2 image_proc owns that
job. The node only checks that the raw and rectified streams are consistent and
saves one evidence pair.
"""

from pathlib import Path

import cv2
import rclpy
from cv_bridge import CvBridge
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from sensor_msgs.msg import CameraInfo, Image

class CameraRectificationCheck(Node):
    def __init__(self):
        super().__init__("camera_rectification_check")

        self.declare_parameter("raw_topic", "/top_camera/image")
        self.declare_parameter("rectified_topic", "/top_camera/image_rect")
        self.declare_parameter("camera_info_topic", "/top_camera/camera_info")
        self.declare_parameter(
            "save_dir", str(Path.home() / "team7_rectification_evidence")
        )
        self.declare_parameter("save_first_pair", True)

        self.bridge = CvBridge()
        self.raw_frames = {}
        self.camera_info = None
        self.logged_info = False
        self.saved_pair = False

        save_dir = Path(self.get_parameter("save_dir").value)
```

```

save_dir.mkdir(parents=True, exist_ok=True)
self.save_dir = save_dir

self.create_subscription(
    Image,
    self.get_parameter("raw_topic").value,
    self.on_raw,
    qos_profile_sensor_data,
)
self.create_subscription(
    Image,
    self.get_parameter("rectified_topic").value,
    self.on_rectified,
    qos_profile_sensor_data,
)
self.create_subscription(
    CameraInfo,
    self.get_parameter("camera_info_topic").value,
    self.on_camera_info,
    qos_profile_sensor_data,
)

self.get_logger().info(
    "rectification checker ready: "
    f"raw={self.get_parameter('raw_topic').value}, "
    f"rectified={self.get_parameter('rectified_topic').value}"
)

@staticmethod
def stamp_key(msg):
    return (msg.header.stamp.sec, msg.header.stamp.nanosec)

def on_camera_info(self, msg):
    self.camera_info = msg
    if self.logged_info:
        return

    self.logged_info = True
    d_values = [float(v) for v in msg.d]
    zero_distortion = all(abs(v) < 1.0e-12 for v in d_values)

    self.get_logger().info(
        f"CameraInfo: {msg.width}x{msg.height}, "
        f"model={msg.distortion_model}, D={d_values}"
    )
    self.get_logger().info(
        "K: "
        f"fx={msg.k[0]:.6f}, fy={msg.k[4]:.6f}, "
        f"cx={msg.k[2]:.6f}, cy={msg.k[5]:.6f}"
    )
    self.get_logger().info(
        "P: "
        f"fx={msg.p[0]:.6f}, fy={msg.p[5]:.6f}, "
        f"cx={msg.p[2]:.6f}, cy={msg.p[6]:.6f}"
    )
    if zero_distortion:
        self.get_logger().info(
            "All distortion coefficients are zero; raw and rectified "
            "Gazebo images are expected to be visually identical."
        )

def on_raw(self, msg):
    try:
        image = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    except Exception as exc:
        self.get_logger().error(f"raw cv_bridge conversion failed: {exc}")
        return

    key = self.stamp_key(msg)

```

```

self.raw_frames[key] = image

# Keep only a small timestamp cache.
if len(self.raw_frames) > 12:
    oldest = sorted(self.raw_frames.keys())[0]
    self.raw_frames.pop(oldest, None)

def on_rectified(self, msg):
    key = self.stamp_key(msg)
    raw = self.raw_frames.pop(key, None)
    if raw is None:
        return

    try:
        rect = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    except Exception as exc:
        self.get_logger().error(
            f"rectified cv_bridge conversion failed: {exc}"
        )
        return

    if raw.shape != rect.shape:
        self.get_logger().error(
            f"shape mismatch: raw={raw.shape}, rectified={rect.shape}"
        )
        return

    diff = cv2.absdiff(raw, rect)
    mean_abs_diff = float(diff.mean())
    max_abs_diff = int(diff.max())

    self.get_logger().info(
        f"matched frame {key[0]}_{key[1]:09d}: "
        f"shape={raw.shape}, mean_abs_diff={mean_abs_diff:.4f}, "
        f"max_abs_diff={max_abs_diff}"
    )

    if self.get_parameter("save_first_pair").value and not self.saved_pair:
        cv2.imwrite(str(self.save_dir / "raw.png"), raw)
        cv2.imwrite(str(self.save_dir / "rectified.png"), rect)
        cv2.imwrite(str(self.save_dir / "difference.png"), diff)
        self.saved_pair = True
        self.get_logger().info(
            f"saved evidence images to {self.save_dir}"
        )

def main():
    rclpy.init()
    node = CameraRectificationCheck()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.try_shutdown()

if __name__ == "__main__":
    main()

```

10.1 Where to put the helper

```

[text]
~/team7_project_bundle/team7_perception_ws/src/team7_perception/
team7_perception/camera_rectification_check.py

```

10.2 Optional replacement setup.py

If the helper should be runnable using `ros2 run`, replace the original `setup.py` with the following version. The only functional change is the additional console-script entry point.

```
[python]
from setuptools import find_packages, setup

package_name = "team7_perception"

setup(
    name=package_name,
    version="1.0.0",
    packages=find_packages(exclude=["test"]),
    data_files=[
        (
            "share/ament_index/resource_index/packages",
            [f"resource/{package_name}"],
        ),
        (f"share/{package_name}", ["package.xml"]),
        (f"share/{package_name}/weights", ["weights/yolo11n-seg_best.onnx"]),
    ],
    install_requires=["setuptools"],
    zip_safe=True,
    maintainer="Moksh Bansal",
    maintainer_email="mokshbansal07@gmail.com",
    description=(
        "ONNX (YOLOv11n-seg) phone/battery detector and camera "
        "rectification validation utilities."
    ),
    license="Apache-2.0",
    tests_require=["pytest"],
    entry_points={
        "console_scripts": [
            "onnx_phone_detector = "
            "team7_perception.onnx_phone_detector_node:main",
            "camera_rectification_check = "
            "team7_perception.camera_rectification_check:main",
        ],
    },
)
```

10.3 Rebuild and run the helper

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
colcon build --symlink-install
source install/setup.bash

ros2 run team7_perception camera_rectification_check
```

Expected result in the current Gazebo camera: zero distortion is logged and `mean_abs_diff` should be zero or very small. Evidence PNGs are saved to `~/team7_rectification_evidence` by default.

11. Debugging guide

Use the following branches in order. Do not skip directly to editing Python when the failure is actually in WSL, Gazebo, bridge, calibration or topic wiring.

Symptom	Likely cause	Fix / next check
D1 — <code>ros2</code> : command not found	ROS setup not sourced or Jazzy not installed.	Run <code>source /opt/ros/jazzy/setup.bash</code> . If the file does not exist, install ROS 2 Jazzy first.
D2 — <code>/top_camera/image</code> missing	Simulation camera model or bridge is not running.	Check <code>run_team7_sim.sh</code> output, <code>ros2</code> topic list, <code>gz</code> model <code>--list</code> , and stop/restart the supplied simulation.

Symptom	Likely cause	Fix / next check
D3 — image_proc / rectify_node missing	image_proc package is not installed.	sudo apt install ros-jazzy-image-proc; then ros2 pkg executables image_proc grep rect.
D4 — camera_info missing	Bridge is missing or only the image topic is running.	Do not run rectification without calibration. Verify /top_camera/camera_info has a publisher.
D5 — image_rect exists but no frames	Input topics/remaps wrong, no CameraInfo messages, or QoS mismatch.	Check ros2 topic info -v on all three topics and confirm the rectifier terminal is still alive.
D6 — rectified image looks identical to raw	Current Gazebo D coefficients are all zero.	This is expected. Record D and the near-zero pixel difference as evidence.
D7 — raw/rectified image is black	Rendering problem upstream of rectification.	Fix the Gazebo camera first. The supplied launchers include LIBGL_ALWAYS_SOFTWARE handling for affected VMs.
D8 — cv_bridge conversion exception	Encoding/dependency mismatch.	Inspect image encoding; ensure cv_bridge is installed; keep desired_encoding="bgr8".
D9 — detector still uses /top_camera/image	Started with run_team7_perception.sh or forgot parameter override.	Restart using direct ros2 run command and verify ros2 param get /onnx_phone_detector image_topic.
D10 — boxes shifted after preprocessing	Coordinate recovery/padding mismatch or wrong source image.	Verify inference.py is unchanged; inspect scale, pad_left, pad_top and confirm outputs are bounded by 1280×960.
D11 — colcon build appears to ignore new helper	Old install tree is sourced or setup.py entry point not rebuilt.	Close/re-source terminal, rebuild. If still stale, clean build/install/log and rebuild once.
D12 — very low frame rate	Gazebo 1280×960 software rendering is the bottleneck.	Compare /top_camera/image Hz and image_rect Hz. Rectification cannot exceed the source camera rate.

11.1 Clean rebuild only when needed

```
[bash]
cd ~/team7_project_bundle/team7_perception_ws
rm -rf build install log
colcon build --symlink-install
source install/setup.bash
```

11.2 If the camera topics exist but publish nothing

```
[bash]
ros2 topic info /top_camera/image -v
ros2 topic info /top_camera/camera_info -v
ros2 topic hz /top_camera/image
ros2 topic hz /top_camera/camera_info
```

If publisher count is zero, the issue is upstream. If publisher count is non-zero but hz never prints, inspect the Gazebo/server logs and camera rendering before touching image_proc.

11.3 If CameraInfo is uncalibrated

A CameraInfo with $K[0] = 0$ is effectively uncalibrated. image_proc cannot perform meaningful rectification from invalid calibration. The current Gazebo setup should not have this problem; for a future physical camera, calibrate the camera or load its calibration YAML before declaring CV-12 complete.

12. Recommended four-terminal workflow

This is the simplest reproducible workflow for the assignee.

Terminal 1 — simulation

```
[bash]
bash ~/team7_project_bundle/run_team7_sim.sh
```

Terminal 2 — rectification

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 run image_proc rectify_node --ros-args \
  -r image:=/top_camera/image \
  -r camera_info:=/top_camera/camera_info \
  -r image_rect:=/top_camera/image_rect
```

Terminal 3 — detector on rectified input

```
[bash]
source /opt/ros/jazzy/setup.bash
cd ~/team7_project_bundle/team7_perception_ws
source install/setup.bash
ros2 run team7_perception onnx_phone_detector --ros-args \
  -p image_topic:=/top_camera/image_rect \
  -p camera_info_topic:=/top_camera/camera_info
```

Terminal 4 — verification/evidence

```
[bash]
source /opt/ros/jazzy/setup.bash
ros2 topic hz /top_camera/image_rect
ros2 param get /onnx_phone_detector image_topic
ros2 topic echo /phone_detections
```

If using the optional helper, run it in Terminal 4 instead and use a fifth terminal for /phone_detections.

13. Evidence worth keeping for CV-12

1. Raw input topic evidence: `ros2 topic info /top_camera/image -v` plus width, height, encoding and header fields from the image topic.
2. CameraInfo evidence: `distortion_model`, `D`, `K` and `P` from `/top_camera/camera_info`.
3. Rectifier running: terminal showing `image_proc rectify_node` with the explicit remappings.
4. Rectified output topic: `ros2 topic info /top_camera/image_rect -v` and `ros2 topic hz output`.
5. Visual evidence: raw and rectified camera images. In Gazebo, note that they are expected to match because `D=0`.
6. Detector input configuration: `ros2 param get /onnx_phone_detector image_topic` showing `/top_camera/image_rect`.
7. Live detector output: `/phone_detections` while the rectified stream is active.
8. Preprocessing code evidence: `inference.py` `letterbox/preprocess/unscale` functions, preferably with a short explanation rather than only a code screenshot.
9. Optional validation evidence: `raw.png`, `rectified.png`, `difference.png` and the `mean_abs_diff` log from `camera_rectification_check.py`.

14. CV-12 completion checklist

- ☐ Ubuntu 24.04 WSL2 + ROS 2 Jazzy starts cleanly.
- ☐ Team 7 Gazebo simulation starts and the overhead camera is visible.
- ☐ `/top_camera/image` is live.
- ☐ `/top_camera/camera_info` is live.
- ☐ Current 1280×960 source resolution is confirmed.
- ☐ CameraInfo width/height match the image.
- ☐ `K` is valid and the distortion model/`D` values are recorded.
- ☐ `image_proc rectify_node` starts with explicit remappings.

- ☐ /top_camera/image_rect is live.
- ☐ Rectified image rate is consistent with the source camera rate.
- ☐ The Gazebo raw/rectified similarity is correctly explained by zero distortion.
- ☐ cv_bridge successfully converts the rectified ROS Image to BGR8.
- ☐ The detector consumes /top_camera/image_rect.
- ☐ ONNX preprocessing still produces a $1 \times 3 \times 640 \times 640$ tensor.
- ☐ Detections/masks map back to 1280×960 source-image coordinates.
- ☐ /phone_detections continues to publish.
- ☐ No geometry.py/CV-17 code was overwritten as part of this task.
- ☐ Evidence screenshots/logs are saved and the exact topic mapping is documented.

15. Handover notes and relationship to later CV tasks

```
[text]
CV-12 camera input + rectification + preprocessing verification
|
+--> CV-13 ONNX node integration/parameterisation
+--> CV-15 publish pixel geometry consistently
+--> CV-17 interpret pixel centroid in world coordinates
+--> CV-18 end-to-end Gazebo testing
+--> CV-19 package rectifier + detector into reusable launch/config
```

The main handover information from CV-12 should be the exact raw topic, CameraInfo topic, rectified topic, calibration behaviour, source-image coordinate convention and detector startup parameter. That gives later tasks a stable camera input contract without forcing them to know how the image was rectified internally.

Recommended merge outcome

For the current sprint, keep image_proc as a separate standard ROS node and keep the detector input configurable. Do not duplicate rectification inside inference.py. When CV-19 is implemented, package image_proc + detector startup into one launch/config so nobody has to remember the remapping command manually.

16. Quick command reference

```
[bash]
# Start sim
bash ~/team7_project_bundle/run_team7_sim.sh

# Start rectifier
source /opt/ros/jazzy/setup.bash
ros2 run image_proc rectify_node --ros-args \
  -r image:=/top_camera/image \
  -r camera_info:=/top_camera/camera_info \
  -r image_rect:=/top_camera/image_rect

# Start detector on rectified image
source /opt/ros/jazzy/setup.bash
cd ~/team7_project_bundle/team7_perception_ws
source install/setup.bash
ros2 run team7_perception onnx_phone_detector --ros-args \
  -p image_topic:=/top_camera/image_rect \
  -p camera_info_topic:=/top_camera/camera_info

# Verify
ros2 topic hz /top_camera/image_rect
ros2 param get /onnx_phone_detector image_topic
ros2 topic echo /phone_detections

# Stop simulation
bash ~/team7_project_bundle/team7_sim_mark2_v1.0/scripts/stop_sim.sh
```

